☁ CLOUD DEVOPS ENGINEER

Complete Study Guide

& CloudTask Production Reference

From Zero to Production — Every Tool, Topic & Best Practice

Covers	Linux · Docker · Kubernetes · CI/CD · AWS/GCP · Terraform · Prometheus · Redis · PostgreSQL · GitHub Actions
Project	CloudTask — Distributed Task Queue System (Production-Grade)
Stack	Node.js / Go · PostgreSQL · Redis · Docker · K8s · AWS EKS / GCP GKE
Duration	8-Month Roadmap · 3–5 hrs/day · Interview-Ready by Month 7

Table of Contents

SECTION 1 — CLOUD DEVOPS CAREER ROADMAP

1. What Is a Cloud DevOps Engineer?

A Cloud DevOps Engineer bridges software development and IT operations. They build the infrastructure, pipelines, and systems that allow code to move from a developer's laptop to a production server reliably, fast, and at scale. Unlike a pure developer, they think about uptime, scalability, automation, and observability. Unlike a pure sysadmin, they write code and build pipelines.

1.1 Core Responsibilities

Responsibility Area	What You Actually Do
Infrastructure as Code	Write Terraform/Pulumi to provision cloud resources (VMs, databases, load balancers) declaratively
CI/CD Pipelines	Build GitHub Actions / Jenkins / GitLab CI pipelines that test, build, and deploy automatically on every commit
Containerization	Package applications into Docker containers, push to registries (ECR, GCR), manage multi-container apps
Kubernetes Orchestration	Deploy containers to K8s clusters, configure autoscaling, manage rolling deployments and rollbacks
Observability	Instrument apps with Prometheus metrics, build Grafana dashboards, set up alerting and log aggregation
Security & Secrets	Manage IAM policies, Kubernetes RBAC, secrets management (Vault, AWS Secrets Manager), network policies
Cloud Networking	Configure VPCs, subnets, security groups, load balancers, DNS (Route 53), CDNs
Incident Response	On-call rotations, runbooks, post-mortems, SLO/SLA tracking, capacity planning

1.2 DevOps Engineer Skill Levels

Level	Experience	Key Skills	Approx. CTC (India)
Junior	0–2 yrs	Linux, Git, Docker basics, one cloud platform, CI/CD basics	8–20 LPA
Mid	2–5 yrs	Kubernetes, Terraform, advanced scripting, monitoring, cloud-native services	20–45 LPA

Level	Experience	Key Skills	Approx. CTC (India)
Senior	5–8 yrs	Multi-cloud, SRE practices, platform engineering, architecture ownership	45–80 LPA
Staff/Lead	8+ yrs	Cloud cost optimization, org-level platform strategy, hiring, vendor management	80–150+ LPA

SECTION 2 — COMPLETE LEARNING ROADMAP

2. The Complete Learning Path

This is the complete skill tree for a Cloud DevOps Engineer. Follow it in order — each layer builds on the previous one. Do NOT skip ahead to Kubernetes without mastering Linux and Docker first.

2.1 Foundation Layer (Month 1–2)

Linux & Shell

The operating system you will spend your entire career working in. Every server, container, and cloud VM runs Linux.

Topic	What to Learn + Commands
Filesystem & Navigation	ls -la, cd, pwd, find, locate, tree. Understand / structure: /etc, /var, /opt, /proc, /sys, /tmp
File Operations	cat, less, head, tail -f (live logs), cp, mv, rm -rf, chmod 755, chown, ln -s (symlinks)
Process Management	ps aux, top, htop, kill -9, pkill, systemctl start/stop/status, journalctl -u service
Networking	curl -X POST, wget, netstat -tlnp, ss -tulnp, nslookup, dig, ping, traceroute, iptables -L
Text Processing	grep -r 'pattern', awk '{print $1}', sed 's/old/new/g', cut -d':' -f1, sort | uniq -c | sort -rn
Disk & Memory	df -h, du -sh *, free -h, lsblk, mount, fdisk -l, /proc/meminfo, vmstat
Users & Permissions	adduser, usermod -aG, /etc/passwd, /etc/sudoers, ssh-keygen, ssh-copy-id, ~/.ssh/authorized_keys
Shell Scripting	#!/bin/bash, variables, if/else, for loops, while, functions, $?, exit codes, cron jobs via crontab -e

RESOURCE: The Linux Command Line (William Shotts) — free at linuxcommand.org. Read chapters 1–12.

PRACTICE: Set up Ubuntu 22.04 on WSL2 (Windows) or UTM (Mac). Use it for everything. No more PowerShell.

CHECKPOINT: Can you write a bash script that monitors a log file and sends an alert when 'ERROR' appears 5 times in 1 min?

Networking Fundamentals

Concept	Description & Why It Matters
OSI Model	7 layers: Physical, Data Link, Network, Transport, Session, Presentation, Application. Docker and K8s networking maps to L3/L4/L7
TCP/IP & UDP	TCP: reliable, ordered, 3-way handshake (SYN-SYN/ACK-ACK). UDP: fast, unreliable. Know which protocols use which
DNS	A record (IP), CNAME (alias), NS (nameserver), TTL. How DNS resolution works: browser → OS cache → resolver → root → TLD → authoritative
HTTP/HTTPS	Methods (GET/POST/PUT/PATCH/DELETE), status codes (2xx/3xx/4xx/5xx), headers, TLS handshake, certificates
Ports	Well-known: SSH=22, HTTP=80, HTTPS=443, PostgreSQL=5432, Redis=6379, Prometheus=9090, Grafana=3000
CIDR & Subnetting	10.0.0.0/16 = 65536 IPs, 10.0.1.0/24 = 256 IPs. VPC design: public subnets (web), private subnets (DB, workers)
Load Balancers	L4 (TCP/UDP — AWS NLB) vs L7 (HTTP — AWS ALB). Health checks, sticky sessions, connection draining
Firewalls & Security Groups	Stateful vs stateless. Inbound/outbound rules. Principle of least privilege. 0.0.0.0/0 on port 22 = bad practice

Git & Version Control

Command/Concept	Usage & Notes
git init / clone	Start a repo or clone. Always use SSH for auth: git clone git@github.com:user/repo.git
Branching Strategy	GitFlow: main (prod), develop (integration), feature/xxx, hotfix/xxx. Trunk-based: short-lived feature branches → main
git rebase vs merge	merge: preserves history, creates merge commit. rebase: linear history, rewrites commits. Use rebase for feature → main PRs
git stash	Temporarily save uncommitted changes. git stash pop to restore. Useful when switching contexts
git bisect	Binary search for the commit that introduced a bug. Critical for debugging production regressions
git hooks	Pre-commit: run linting/tests before commit. Pre-push: run full test suite. Used in CI/CD locally
Conventional Commits	feat: / fix: / docs: / test: / ci: / perf: / refactor: / chore: — enables automated changelogs and semantic versioning
Protected Branches	Require PR + review + CI pass before merge. No direct push to main. Enable in GitHub Settings → Branches

2.2 Containerization (Month 2–3)

Docker — Complete Reference

Docker packages an application and all its dependencies into a container that runs identically everywhere. The most critical tool in modern DevOps.

Docker Architecture

Component	Description
Docker Daemon (dockerd)	Background service that manages containers, images, networks, volumes. Client communicates via REST API
Docker Image	Read-only template with layers. Each instruction in Dockerfile = one layer. Layers are cached and shared
Docker Container	Running instance of an image. Has its own PID namespace, network namespace, filesystem (writable layer on top)
Docker Registry	Store for images. Docker Hub (public), ECR (AWS), GCR (GCP), GHCR (GitHub). Push/pull like git
Docker Network	bridge (default, containers on same host), host (share host network), overlay (multi-host, swarm/K8s), none
Docker Volume	Persistent storage outside the container filesystem. Survives container restarts and deletions

Writing Production Dockerfiles

A bad Dockerfile creates oversized, insecure images. A good Dockerfile creates small, reproducible, secure images.

```
PRODUCTION DOCKERFILE PATTERN — CloudTask API


# Stage 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
```

```
# Stage 2: Runtime — minimal image

FROM node:20-alpine

RUN addgroup -S appgroup && adduser -S appuser -G appgroup

WORKDIR /app

COPY --from=builder /app/node_modules ./node_modules

COPY --from=builder /app/src ./src

USER appuser

EXPOSE 3000

HEALTHCHECK --interval=30s --timeout=3s CMD wget -qO-
http://localhost:3000/api/v1/health || exit 1

CMD ['node', 'src/index.js']
```

Best Practice	Why It Matters
Use alpine base images	node:20 = 900MB. node:20-alpine = 50MB. Smaller = faster pull, less attack surface
Multi-stage builds	Build stage has compilers/dev tools. Runtime stage has only production artifacts. Prevents leaking build secrets
Non-root user	Default containers run as root — if exploited, attacker has host root. Add dedicated user for security
Copy package.json first	Docker layer cache: if package.json unchanged, npm install layer is reused. Speeds up builds 10x
Use .dockerignore	Exclude node_modules, .git, .env, test files. Prevents secrets from leaking into image and reduces context size
HEALTHCHECK instruction	K8s/Docker Swarm uses this to know if container is healthy. Without it, a crashed app keeps receiving traffic
Pin image versions	FROM node:20.11-alpine3.19 not FROM node:latest. Reproducible builds. latest breaks unexpectedly

Docker Compose — Local Development

Docker Compose runs your entire stack (API + Worker + PostgreSQL + Redis) with one command. This is how CloudTask dev environment works.

```
docker-compose.yml — CloudTask Local Stack


version: '3.8'
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: cloudtask
      POSTGRES_USER: cloudtask
      POSTGRES_PASSWORD: ${DB_PASSWORD}   # from .env file
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U cloudtask']
      interval: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    command: redis-server --requirepass ${REDIS_PASSWORD}
    healthcheck:
      test: ['CMD', 'redis-cli', 'ping']

  api:
    build: ./api
    ports: ['3000:3000']
```

```
    environment:
      DATABASE_URL: postgresql://cloudtask:${DB_PASSWORD}@postgres:5432/cloudtask
      REDIS_URL: redis://:${REDIS_PASSWORD}@redis:6379
      JWT_SECRET: ${JWT_SECRET}
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_healthy }

  worker:
    build: ./worker
    environment:
      DATABASE_URL: postgresql://cloudtask:${DB_PASSWORD}@postgres:5432/cloudtask
      REDIS_URL: redis://:${REDIS_PASSWORD}@redis:6379
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_healthy }
    deploy:
      replicas: 2

volumes:
  postgres_data:
```

Essential Docker Commands

Command	What It Does
docker build -t cloudtask/api:1.0.0 .	Build image, tag it with name:version

Command	What It Does
docker run -d -p 3000:3000 --name api cloudtask/api:1.0.0	Run container in background, map port, name it
docker exec -it api sh	Get a shell inside running container. -it = interactive + TTY
docker logs -f api	Stream logs from container. -f = follow (like tail -f)
docker inspect api	Full JSON metadata about container: IP, mounts, env vars, health status
docker stats	Live CPU/memory/network usage of all running containers
docker system prune -af	Delete all stopped containers, unused images, networks. Free disk space. Dangerous — don't run on prod
docker image scan cloudtask/api:1.0.0	Scan for CVEs using Snyk/Docker Scout. Always scan before pushing to production

2.3 Kubernetes (Month 3–5)

Kubernetes (K8s) is the container orchestration platform that runs your containers at scale, handles failures, manages rolling deployments, and auto-scales your services. The most important technology in modern cloud infrastructure.

Core K8s Objects

Object	Description + CloudTask Usage
Pod	Smallest deployable unit. 1+ containers sharing a network namespace and storage. A pod running your API container
Deployment	Manages a set of identical pods. Handles rolling updates, rollbacks, scaling. CloudTask API has a Deployment with 3 replicas
ReplicaSet	Created automatically by Deployment to maintain the desired number of pod replicas. Usually don't interact directly
Service	Stable network endpoint for pods. Types: ClusterIP (internal), NodePort (external via node IP), LoadBalancer (cloud LB)
Ingress	L7 HTTP routing rules. Routes api.cloudtask.com → API service, grafana.cloudtask.com → Grafana service. Uses nginx-ingress
ConfigMap	Store non-sensitive config as key-value pairs. Mounted as env vars or files. Example: feature flags, app config
Secret	Base64-encoded sensitive data. Mounted as env vars. In production: use External Secrets Operator with AWS Secrets Manager
PersistentVolume (PV)	A piece of storage provisioned in the cluster. CloudTask: PostgreSQL data stored on AWS EBS via PersistentVolumeClaim
HorizontalPodAutoscaler	Scales replicas based on CPU/memory or custom metrics (queue depth via KEDA). CloudTask worker scales 1→20 pods
Namespace	Virtual cluster within a cluster. Separate cloudtask-prod, cloudtask-staging, monitoring namespaces
ServiceAccount	Identity for pods to authenticate to K8s API and cloud services (via IRSA on AWS). Principle of least privilege
NetworkPolicy	Firewall rules for pod-to-pod traffic. Allow: worker → postgres. Deny: internet → postgres directly
LimitRange	Set default CPU/memory requests and limits per pod in a namespace. Prevents resource hogging
ResourceQuota	Total resource cap per namespace. Prevents one team from consuming all cluster resources

K8s Manifests — CloudTask Production

These are the actual YAML files you write. Every field matters.

API Deployment — infra/k8s/api-deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cloudtask-api
  namespace: cloudtask-prod
  labels:
    app: cloudtask-api
    version: '1.0.0'
spec:
  replicas: 3
  selector:
    matchLabels:
      app: cloudtask-api
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1        # 1 extra pod during update
      maxUnavailable: 0  # 0 pods down during update
  template:
    metadata:
      labels:
        app: cloudtask-api
    spec:
```

```
      containers:
      - name: api
        image: ghcr.io/youruser/cloudtask-api:SHA_HASH
        ports:
        - containerPort: 3000
        envFrom:
        - secretRef:
            name: cloudtask-secrets
        resources:
          requests:
            cpu: 100m    # 0.1 CPU cores
            memory: 128Mi
          limits:
            cpu: 500m    # 0.5 CPU cores
            memory: 512Mi
        readinessProbe:
          httpGet:
            path: /api/v1/health
            port: 3000
          initialDelaySeconds: 5
          periodSeconds: 10
        livenessProbe:
          httpGet:
            path: /api/v1/health
            port: 3000
          initialDelaySeconds: 15
```

```
        periodSeconds: 30
```

```
Worker HPA with KEDA — infra/k8s/hpa.yaml


apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: cloudtask-worker-scaler
  namespace: cloudtask-prod
spec:
  scaleTargetRef:
    name: cloudtask-worker
  minReplicaCount: 1
  maxReplicaCount: 20
  cooldownPeriod: 60
  triggers:
  - type: redis
    metadata:
      address: redis.cloudtask-prod.svc.cluster.local:6379
      listName: task_queue    # Redis list being monitored
      listLength: '5'         # Scale up if > 5 items per replica
      enableTLS: 'false'
    authenticationRef:
      name: redis-trigger-auth
```

K8s Networking Deep Dive

Concept	How It Works
kube-proxy	Runs on every node. Implements Service networking via iptables/IPVS rules. Routes traffic to pod IPs
CoreDNS	K8s internal DNS. cloudtask-api.cloudtask-prod.svc.cluster.local resolves to the Service ClusterIP
CNI Plugin	Container Network Interface: Calico (network policies), Flannel (simple), Cilium (eBPF-based, best performance)
Service Types	ClusterIP: internal only. NodePort: external via node IP:port. LoadBalancer: cloud LB. ExternalName: DNS alias
Ingress Controller	nginx-ingress or AWS ALB Controller. Reads Ingress resources, configures reverse proxy routing rules
Pod-to-Pod Traffic	Pods get routable IPs. Direct communication within cluster without NAT. NetworkPolicies control this

2.4 CI/CD Pipelines (Month 4–5)

Continuous Integration (CI) automatically tests and builds code on every commit. Continuous Delivery/Deployment (CD) automatically deploys tested code to production. Together they eliminate manual deployment steps and make releasing code safe and fast.

GitHub Actions — Complete Reference

Concept	Description
Workflow	A YAML file in .github/workflows/. Defines when to run (trigger) and what to do (jobs). Can have multiple workflows
Trigger (on:)	push: (any push), pull_request: (PR events), schedule: (cron), workflow_dispatch: (manual button in GitHub UI)
Job	A set of steps that run on a Runner (ubuntu-latest, macos-latest, windows-latest, or self-hosted). Jobs run in parallel by default
Step	A single task: run a command or use an action. Steps within a job run sequentially
Action	Reusable unit of work from GitHub Marketplace. actions/checkout@v4, docker/login-action, aws-actions/configure-aws-credentials
Secrets	Stored encrypted in GitHub Settings. Access via ${{ secrets.MY_SECRET }}. Never log secrets or put in code
Matrix Strategy	Run same job with different variables: [node-version: [18, 20], os: [ubuntu, macos]]. 4 parallel runs from 1 job definition
Caching	actions/cache@v3 — cache node_modules, pip packages, Docker layer cache. Can cut pipeline from 5min to 1min
Environments	Define prod/staging environments with protection rules: required reviewers, deployment branch restrictions
OIDC Auth	Keyless auth to AWS/GCP via OpenID Connect. No long-lived access keys stored as secrets — best security practice

```
CloudTask Full CI/CD Pipeline — .github/workflows/deploy.yml


name: CI/CD Pipeline

on:

  push:

    branches: [main]

  pull_request:
```

```
    branches: [main]


jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_DB: test_db, POSTGRES_USER: test,
POSTGRES_PASSWORD: test }
        options: --health-cmd pg_isready
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm run lint
      - run: npm test
        env: { DATABASE_URL: postgresql://test:test@localhost:5432/test_db }


  build-push:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    permissions:
      contents: read
```

```
      packages: write
      id-token: write   # For OIDC
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with: { registry: ghcr.io, username: ${{ github.actor }}, password:
${{ secrets.GITHUB_TOKEN }} }
      - uses: docker/build-push-action@v5
        with:
          push: true
          tags: ghcr.io/${{ github.repository }}/api:${{ github.sha }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

  deploy:
    needs: build-push
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: aws-actions/configure-aws-credentials@v4
        with: { role-to-assume: ${{ secrets.AWS_ROLE_ARN }}, aws-region: ap-south-1 }
      - run: aws eks update-kubeconfig --name cloudtask-cluster --region ap-south-1
      - run: kubectl set image deployment/cloudtask-api api=ghcr.io/.../api:${{ github.sha }}
      - run: kubectl rollout status deployment/cloudtask-api --timeout=5m
```

2.5 Cloud Platforms — AWS & GCP (Month 4–6)

AWS Services for CloudTask

Service	Purpose & CloudTask Usage
EKS (Elastic Kubernetes Service)	Managed K8s control plane. Run CloudTask API + Worker. Cost: $0.10/hr for control plane + EC2 nodes
ECR (Elastic Container Registry)	Store Docker images. Private registry integrated with EKS via IAM. Alternative to Docker Hub (no rate limits)
RDS PostgreSQL	Managed PostgreSQL. Auto-backups, Multi-AZ for HA, automated patching. Free tier: db.t3.micro 20GB
ElastiCache Redis	Managed Redis. CloudTask task queue lives here. Automatic failover with Redis replication groups
ALB (Application Load Balancer)	L7 load balancer in front of EKS. Routes to K8s Ingress. SSL termination, WAF integration, access logs to S3
VPC (Virtual Private Cloud)	Isolated network. Public subnets: ALB. Private subnets: EKS nodes, RDS, ElastiCache. No direct internet access to DB
IAM (Identity & Access Management)	Roles for EKS worker nodes, IRSA for pods, CI/CD OIDC role. Principle of least privilege always
AWS Secrets Manager	Store DB passwords, JWT secrets. K8s External Secrets Operator syncs to K8s Secrets automatically
CloudWatch	Logs from EKS (Container Insights), ALB access logs, RDS slow query logs. Metrics and alerting
Route 53	DNS for api.cloudtask.com pointing to ALB. Health checks, failover routing, latency-based routing
S3	Store load test results, Terraform state backend, application artifacts, CloudWatch log exports
ACM (Certificate Manager)	Free SSL/TLS certificates. Attach to ALB for HTTPS. Auto-renews. Never manage certs manually

AWS Free Tier Strategy (₹0 Deployment)

KEEP COSTS ZERO while building production skills:

EKS Alternative: Use k3s on free-tier EC2 t2.micro (OR) use minikube locally, OR use GKE Autopilot free tier ($300 credit)

Database: Use Railway.app free tier for PostgreSQL (500MB free) OR Supabase free tier

Redis: Use Upstash free tier (10,000 requests/day) — perfect for development

Container Registry: Use GHCR (GitHub Container Registry) — free for public repos

Domain: Use nip.io for fake domains (e.g., 1.2.3.4.nip.io routes to 1.2.3.4) — no DNS needed

Monitoring: Run Prometheus + Grafana locally via docker-compose — no cloud cost

CI/CD: GitHub Actions 2000 free minutes/month — more than enough

2.6 Infrastructure as Code — Terraform (Month 5–6)

Terraform lets you define cloud infrastructure in code. Instead of clicking through AWS console, you write .tf files. Run terraform apply and it creates your VPC, EKS cluster, RDS instance. Version controlled, reproducible, and auditable.

Core Terraform Concepts

Concept	Description
Provider	Plugin for a cloud platform. hashicorp/aws, hashicorp/google, hashicorp/kubernetes. Configured with credentials
Resource	A cloud object to create. aws_eks_cluster, aws_db_instance, google_container_cluster
Module	Reusable group of resources. terraform-aws-modules/eks/aws creates full EKS setup in 20 lines
State	terraform.tfstate tracks what Terraform has created. Store in S3 + DynamoDB (locking) for teams. NEVER commit to git
Plan	terraform plan shows what will be created/changed/destroyed BEFORE applying. Always review, always use -out
Variables	Input variables (var.region), output values (output.cluster_endpoint), locals (local.common_tags)
Data Sources	Query existing resources not managed by Terraform. data.aws_ami.ubuntu gets latest Ubuntu AMI ID
Workspace	Separate state per environment (dev/staging/prod) using same code. terraform workspace new staging

```
CloudTask EKS Cluster — infra/terraform/main.tf (simplified)


terraform {
  required_version = '>= 1.6'
  backend 's3' {
    bucket         = 'cloudtask-tfstate'
    key            = 'prod/terraform.tfstate'
    region         = 'ap-south-1'
    dynamodb_table = 'terraform-locks'  # Prevents concurrent applies
  }
```

```
}


module 'eks' {
  source  = 'terraform-aws-modules/eks/aws'
  version = '~> 20.0'
  cluster_name    = 'cloudtask-prod'
  cluster_version = '1.29'
  vpc_id          = module.vpc.vpc_id
  subnet_ids      = module.vpc.private_subnets
  eks_managed_node_groups = {
    workers = {
      min_size       = 1
      max_size       = 5
      instance_types = ['t3.medium']
    }
  }
}
```

2.7 Observability — Prometheus + Grafana (Month 6–7)

Observability is your ability to understand what your system is doing at any point in time. The three pillars are Metrics, Logs, and Traces. You cannot run production systems without these.

The Three Pillars of Observability

Pillar	Tool	What It Captures
Metrics	Prometheus + Grafana	Numeric measurements over time: request rate, error rate, latency P99, CPU%, queue depth
Logs	Loki / CloudWatch / ELK	Event records with timestamp and context: error messages, audit trails, debug info
Traces	Jaeger / Zipkin / AWS X-Ray	Request lifecycle across services: API → Worker → DB query timing, where latency comes from

Prometheus Metrics Types

Metric Type	Description + CloudTask Example
Counter	Only goes up. Total number of completed tasks, total API requests, total errors. Use rate(counter[5m]) for per-second rate
Gauge	Goes up and down. Current queue depth, current active workers, current memory usage
Histogram	Samples observations into configurable buckets. API request duration: ≤10ms, ≤50ms, ≤200ms, ≤1s. Enables P50/P95/P99
Summary	Similar to histogram but percentiles calculated client-side. Less flexible than histogram for aggregation across instances

Instrumenting CloudTask API (Node.js)

```
Prometheus instrumentation — api/src/middleware/metrics.js


const client = require('prom-client');
const register = client.register;
client.collectDefaultMetrics({ register });  // CPU, memory, event loop
```

```
const httpRequestDuration = new client.Histogram({
  name: 'http_request_duration_seconds',
  help: 'Duration of HTTP requests in seconds',
  labelNames: ['method', 'route', 'status_code'],
  buckets: [0.01, 0.05, 0.1, 0.2, 0.5, 1, 2, 5],
});

const tasksSubmitted = new client.Counter({
  name: 'cloudtask_tasks_submitted_total',
  help: 'Total tasks submitted',
  labelNames: ['task_type', 'priority'],
});

const queueDepth = new client.Gauge({
  name: 'cloudtask_queue_depth',
  help: 'Current number of tasks in Redis queue',
});

// In your route handler:
const end = httpRequestDuration.startTimer({ method: req.method, route: req.path });
// ... handle request ...
end({ status_code: res.statusCode });

// /metrics endpoint:
app.get('/api/v1/metrics', async (req, res) => {
  res.set('Content-Type', register.contentType);
```

```
  res.send(await register.metrics());
});
```

Key PromQL Queries for CloudTask

PromQL Query	What It Measures
rate(http_request_duration_seconds_count[5m])	API requests per second over last 5 minutes
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))	P99 latency — 99% of requests complete in under this time
sum(rate(http_requests_total{status_code=~'5..'}[5m])) / sum(rate(http_requests_total[5m]))	Error rate as a ratio (5xx errors / total requests)
cloudtask_queue_depth	Current Redis queue depth — trigger scaling alerts here
rate(cloudtask_tasks_processed_total{status='done'}[5m])	Worker throughput: tasks successfully processed per second
process_resident_memory_bytes / 1024 / 1024	RSS memory usage in MB — watch for memory leaks

2.8 Redis — Complete Reference

Redis is an in-memory data structure store used as a database, cache, and message broker. In CloudTask it serves as the task queue (Redis Lists) and rate limiting store.

Redis Data Structures

Data Type	Commands & CloudTask Usage
String	SET key value EX 3600 (expire in 1hr). GET key. INCR counter. Used for: rate limiting counters, session tokens, caching
List	LPUSH queue_name task_json (add to left/head). BRPOP queue_name 0 (blocking pop from right/tail with timeout). CloudTask TASK QUEUE uses this
Hash	HSET user:123 name 'Alice' email 'alice@example.com'. HGET user:123 name. HGETALL user:123. Store structured objects
Set	SADD online_users user123. SISMEMBER online_users user123. SCARD online_users. Used for: unique visitor tracking, tags
Sorted Set	ZADD leaderboard 1500 'Alice'. ZRANGE leaderboard 0 9 WITHSCORES REV. Priority queues, leaderboards, rate limiting with timestamps
Pub/Sub	SUBSCRIBE events. PUBLISH events 'task_completed'. Real-time notifications. CloudTask: notify clients of task completion
Streams	XADD stream * key value. XREAD COUNT 10 STREAMS stream 0. Like Kafka but simpler. Persistent, consumer groups supported

CloudTask Queue Implementation

```
Reliable Queue Pattern — Using RPOPLPUSH for at-least-once delivery



// PRODUCER (API server) — api/src/services/queue.js

async function enqueueTask(taskId, taskType, payload, priority) {

  const job = JSON.stringify({ taskId, taskType, payload, priority, enqueuedAt:
Date.now() });

  const queueName = priority <= 3 ? 'task_queue:high' : 'task_queue:normal';

  await redis.lpush(queueName, job);  // Add to head of list

  await redis.incr('metrics:tasks_enqueued');
```

```
}

// CONSUMER (Worker) — worker/src/queue/consumer.js
async function processLoop() {
  while (true) {
    // BRPOP blocks until an item is available (no CPU waste)
    // Check high priority queue first, then normal
    const result = await redis.brpop('task_queue:high', 'task_queue:normal', 0);
    if (!result) continue;

    const [queueName, jobData] = result;
    const job = JSON.parse(jobData);

    // Move to processing queue for crash recovery
    await redis.lpush('task_queue:processing', jobData);

    try {
      await processJob(job);
      await redis.lrem('task_queue:processing', 1, jobData);  // Remove on success
    } catch (err) {
      await handleFailedJob(job, err, jobData);
    }
  }
}
```

Rate Limiting with Redis — Sliding Window

```
// 100 requests per minute per user — sliding window algorithm
async function checkRateLimit(userId) {
  const key = `ratelimit:${userId}:${Math.floor(Date.now() / 60000)}`;
  const current = await redis.incr(key);
  if (current === 1) await redis.expire(key, 120);  // 2 min TTL
  if (current > 100) {
    throw new Error('Rate limit exceeded — 100 requests per minute');
  }
}
```

2.9 PostgreSQL — Production Deep Dive

Essential SQL for CloudTask

```
-- Full CloudTask schema with indexes
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE tasks (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  type        VARCHAR(50) NOT NULL,
  payload     JSONB NOT NULL,
  status      VARCHAR(20) NOT NULL DEFAULT 'pending'
              CHECK (status IN ('pending','running','done','failed','dead')),
  priority    INT NOT NULL DEFAULT 5 CHECK (priority BETWEEN 1 AND 10),
  attempts    INT NOT NULL DEFAULT 0,
  max_retries INT NOT NULL DEFAULT 3,
  error_msg   TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  started_at  TIMESTAMPTZ,
  finished_at TIMESTAMPTZ
);
```

```
-- Indexes (critical for query performance)

CREATE INDEX idx_tasks_status ON tasks(status);

CREATE INDEX idx_tasks_user_id ON tasks(user_id);

CREATE INDEX idx_tasks_created_at ON tasks(created_at DESC);

CREATE INDEX idx_tasks_status_priority ON tasks(status, priority) WHERE status = 'pending';
```

PostgreSQL Internals — What Interviewers Ask

Topic	What to Know
ACID Transactions	Atomicity (all or nothing), Consistency (constraints maintained), Isolation (concurrent transactions don't interfere), Durability (committed data persists even after crash)
MVCC	Multi-Version Concurrency Control. Readers don't block writers. Each transaction sees a snapshot of data. Old versions cleaned by VACUUM
Isolation Levels	READ UNCOMMITTED → READ COMMITTED → REPEATABLE READ → SERIALIZABLE. Higher = less concurrency. PostgreSQL default: READ COMMITTED
B-Tree Index	Default index type. O(log n) lookups. Good for =, <, >, BETWEEN, LIKE 'prefix%'. Not for LIKE '%suffix'
EXPLAIN ANALYZE	Run EXPLAIN ANALYZE SELECT... to see query execution plan, actual time, rows. Look for Seq Scan on large tables — add index
Connection Pooling	PostgreSQL spawns a process per connection (expensive). Use PgBouncer (transaction mode) to pool connections. Critical at scale
WAL (Write-Ahead Log)	All changes written to WAL first. Basis for crash recovery, replication, and logical decoding (CDC). WAL shipping = streaming replication
Partitioning	Range partition tasks table by created_at month. Old partitions can be archived or dropped without full table scan

SECTION 3 — CLOUDTASK PROJECT: PRODUCTION IMPLEMENTATION

3. CloudTask: Architecture & Implementation

CloudTask is your production reference project. Every concept you learn in Section 2 gets applied here. This section gives you the complete implementation reference.

3.1 System Architecture

Component	Technology & Role
API Server	Node.js (Express) — accepts task submissions, JWT auth, rate limiting, exposes /metrics endpoint
Worker Service	Node.js or Go — polls Redis queue (BRPOP), executes jobs, updates PostgreSQL, implements retry logic
Task Queue	Redis Lists (LPUSH/BRPOP) — two queues: task_queue:high and task_queue:normal. FIFO per priority
Database	PostgreSQL — tasks table (UUID PKs, JSONB payload, status FSM), users table, indexes on status/user_id
Scheduler	Node-cron in a separate container — pushes recurring/delayed tasks to Redis at the right time
Monitoring	Prometheus scrapes /metrics every 15s. Grafana dashboards. Alerts via AlertManager or Grafana alerts
Ingress	nginx-ingress controller on K8s. Routes api.cloudtask.com/api → API service, /grafana → Grafana service
CI/CD	GitHub Actions: test → build → push GHCR → kubectl rollout on merge to main

3.2 Retry & Dead Letter Queue

```
Exponential Backoff with Jitter — worker/src/retry/backoff.js


function calculateBackoff(attempts) {
  const baseDelay = 1000;  // 1 second
  const maxDelay = 30000;  // 30 seconds cap
```

```
  // Exponential: 1s, 2s, 4s, 8s... with random jitter
  // Jitter prevents 'thundering herd' — all workers retrying simultaneously
  const exponential = Math.min(baseDelay * Math.pow(2, attempts), maxDelay);
  const jitter = Math.random() * exponential * 0.2;  // ±20% randomness
  return exponential + jitter;
}


async function handleFailedJob(job, error, rawJobData) {
  job.attempts += 1;
  if (job.attempts >= job.maxRetries) {
    // Move to dead letter queue
    await redis.lpush('task_queue:dead', JSON.stringify({ ...job, finalError:
error.message }));
    await db.query('UPDATE tasks SET status=$1, error_msg=$2 WHERE id=$3', ['dead',
error.message, job.taskId]);
    logger.error({ taskId: job.taskId, error: error.message }, 'Task moved to DLQ');
  } else {
    // Requeue with delay
    const delay = calculateBackoff(job.attempts);
    await db.query('UPDATE tasks SET status=$1, attempts=$2, error_msg=$3 WHERE
id=$4',
      ['pending', job.attempts, error.message, job.taskId]);
    setTimeout(async () => {
      await redis.lpush('task_queue:normal', JSON.stringify(job));
    }, delay);
    logger.warn({ taskId: job.taskId, attempt: job.attempts, retryIn: delay }, 'Retrying task');
  }
```

```
}
```

3.3 Phase-by-Phase Build Plan

Phase	Duration	Goal	Deliverable
1	8 weeks	REST API + JWT Auth + PostgreSQL	POST/GET/PATCH /tasks, auth middleware, DB schema, Swagger docs, integration tests
2	8 weeks	Redis Queue + Worker + Docker	Worker processing tasks, docker-compose stack, deployed to cloud URL
3	8 weeks	Kubernetes + Autoscaling + Reliability	K8s manifests, KEDA HPA, retry/DLQ, rate limiting, load test results
4	8 weeks	Observability + CI/CD + Polish	Prometheus + Grafana, GitHub Actions pipeline, v1.0.0 release, interview-ready

3.4 Phase Checkpoint Questions (Must Answer Before Moving On)

Phase 1 — API Checkpoints

- What happens when two requests hit your API at exactly the same time and both try to create a task with the same ID?
- Why did you choose JWT over session-based auth? What are the tradeoffs?
- What is a database index and when did you add one in this project?
- What does your error handler return when someone passes an invalid task ID? Show the response.
- Walk me through exactly what happens from POST /tasks hitting the server to a row appearing in PostgreSQL.

Phase 2 — Queue Checkpoints

- Why use BRPOP instead of polling Redis every second? What is the difference in CPU usage?
- What happens if your worker crashes while processing a task? Is the task lost? How would you fix this?
- Explain the difference between your API Dockerfile and worker Dockerfile. Why is the worker different?
- What would happen if 100 tasks arrive at the same time and you only have one worker?
- How does your worker know which PostgreSQL database to connect to in production vs local dev?

Phase 3 — K8s Checkpoints

- Walk me through your HPA configuration. What metric does it scale on? What are your min/max replicas?
- A task fails 3 times. Walk me through exactly what happens step by step in your system.
- What is jitter in exponential backoff and why does it matter? Show the code.
- What is the difference between a Kubernetes Deployment and a Pod?
- Your API is getting rate-limited requests at 200/min from one user. How does your system handle it?

Phase 4 — Observability Checkpoints

- What is P99 latency? What is the difference between P50, P95, and P99?
- Your Grafana alert fires — queue depth has been > 100 for 10 minutes. Walk me through your debugging process.
- Your CI pipeline fails on a PR. What are the first 3 things you check?
- How would you add a new job type (e.g., send SMS) to this system without breaking existing functionality?
- If you had to scale this to 10x the current load, what would you change first and why?

SECTION 4 — STUDY NOTES: KEY CONCEPTS & INTERVIEW PREP

4. Critical Concepts — Study Notes

4.1 System Design Fundamentals

Concept	Definition + When It Matters
CAP Theorem	A distributed system can provide at most 2 of: Consistency (all nodes see same data), Availability (every request gets a response), Partition Tolerance (works despite network splits). PostgreSQL: CP. Redis: AP. Cassandra: AP
Eventual Consistency	Data will be consistent eventually, but reads may return stale data temporarily. Redis replication is eventually consistent. DynamoDB by default
Idempotency	Calling an operation multiple times produces the same result as calling it once. Critical for retries — ensure tasks have unique IDs and workers check before processing
Horizontal vs Vertical Scaling	Vertical: bigger machine (more RAM/CPU). Horizontal: more machines. CloudTask worker scales horizontally via K8s replicas — stateless design required
Thundering Herd	Many processes wake up simultaneously (e.g., all workers retrying at the same time). Solved by jitter (randomizing retry delays) — implemented in CloudTask backoff
Backpressure	When a consumer is slower than producer, signal the producer to slow down. In CloudTask: if queue > threshold, return 429 to API callers
Fan-out vs Fan-in	Fan-out: 1 event → many queues/consumers (notification service). Fan-in: many producers → 1 queue (CloudTask). Know the pattern for system design interviews
SLA vs SLO vs SLI	SLI: actual metric (latency=150ms). SLO: target (latency<200ms 99% of time). SLA: contract (99.9% uptime or credits). Error budget = 100% - SLO

4.2 Security Best Practices

Security Area	What to Implement
Secrets Management	Never hardcode secrets. Use env vars. In K8s: Kubernetes Secrets + External Secrets Operator syncing from AWS Secrets Manager / HashiCorp Vault

Security Area	What to Implement
JWT Security	Use RS256 (asymmetric) not HS256 (symmetric) in production. Short expiry (15min) + refresh tokens. Validate alg, exp, iss, aud claims
Container Security	Non-root user, read-only filesystem (securityContext.readOnlyRootFilesystem: true), no privileged mode, scan images with Trivy/Snyk before push
K8s RBAC	ServiceAccount per app with minimum permissions. No 'cluster-admin' for workloads. Use NetworkPolicies to restrict pod communication
SQL Injection	Always use parameterized queries (never string concatenation). ORM like Prisma handles this. Run sqlmap in staging to verify
Rate Limiting	Redis sliding window counter per user. Return 429 with Retry-After header. Also: IP-based limiting at Ingress/API Gateway level
TLS Everywhere	HTTPS for all external traffic (cert-manager + Let's Encrypt in K8s). mTLS for internal service-to-service (Istio/Linkerd service mesh)
Dependency Scanning	npm audit / pip check in CI pipeline. Dependabot / Renovate for automated dependency updates. Snyk for runtime scanning

4.3 Debugging Playbook

Symptom	Debug Steps
Pod stuck in CrashLoopBackOff	kubectl describe pod podname → check Events section for error. kubectl logs podname --previous → last container's output before crash
Service unreachable in cluster	kubectl get endpoints svcname → should show pod IPs. If empty, labels on pod don't match selector. kubectl exec -it pod -- nslookup svc
High latency in Grafana	Check P99 histogram. DB slow query log (log_min_duration_statement=1000). Use EXPLAIN ANALYZE. Check connection pool exhaustion
Worker not processing tasks	redis-cli LLEN task_queue — is queue growing? Check worker logs for errors. kubectl get hpa — is it scaling? Check Redis password env var
CI pipeline failing	Read error message in Actions tab. Common: missing env secret, Docker build context wrong path, npm ci fails (package-lock.json stale)
OOM Killed pods	kubectl describe pod → Last State: OOMKilled. Increase memory limit. Check for memory leak: process_resident_memory_bytes Grafana panel
PostgreSQL connection refused	telnet postgres-host 5432. Check security group/NetworkPolicy allows port 5432 from worker subnet. Check pg_hba.conf allows connections

4.4 Interview Quick Reference — Must-Know Answers

THE 2-MINUTE PROJECT PITCH (Memorize this):

'I built CloudTask — a production-grade distributed task queue system.

The API lets users submit background jobs like emails or reports.

A Redis-backed worker service picks up jobs asynchronously, processes them,

and updates a PostgreSQL database. The system auto-scales workers on

Kubernetes based on queue depth using KEDA.

I added exponential backoff retries, a dead letter queue for failed jobs,

JWT auth with rate limiting, and a full Prometheus + Grafana observability stack.

The whole thing deploys via GitHub Actions CI/CD to AWS EKS.

During load testing with k6, it handled 1000 concurrent task submissions

with P99 latency under 200ms.'

Interview Question	Key Points in Your Answer
Why Redis over Kafka?	For this scale, Redis is simpler. Kafka: higher throughput, replay, consumer groups. Redis: lower latency, simpler ops. Mention you know the tradeoff
Why PostgreSQL not MongoDB?	Task state changes need ACID. Running → done cannot be a partial write. MongoDB eventual consistency is wrong for task state machine
How to scale 10x?	Stateless API pods → horizontal scale. KEDA worker autoscale. PgBouncer connection pooling. Redis Cluster. Read replicas for queries. CDN for static
Explain CAP theorem	Pick 2 of 3: Consistency, Availability, Partition Tolerance. In CloudTask: PostgreSQL = CP (consistency). Redis = AP (availability). Network partitions always happen
What is your worst bottleneck?	PostgreSQL single write node. Would add PgBouncer, read replicas, partition tasks table by created_at, eventually move to append-only event log
Explain Docker vs VM	VM: full OS, hypervisor, seconds to start, GBs of RAM. Docker: shared kernel, namespaces+cgroups, milliseconds to start, MBs of RAM. Not a VM replacement — different tool
What is a Kubernetes pod?	Smallest deployable unit in K8s. 1+ containers sharing network namespace (same IP) and storage. All containers in a

Interview Question	Key Points in Your Answer
	pod are co-scheduled on the same node
Explain P99 latency	If P99 = 200ms: 99% of requests complete in under 200ms, 1% take longer. P50 (median) hides outliers. P99 catches slow requests affecting real users

SECTION 5 — RESOURCES, TOOLS & DAILY SCHEDULE

5. Curated Resources

5.1 Free Learning Resources (Priority Order)

Resource	URL	What For	When
Linux Command Line	linuxcommand.org	Linux & shell scripting fundamentals	Month 1
PostgreSQL Tutorial	postgresqltutorial.com	SQL from zero to advanced indexes	Month 1
Node.js Official Docs	nodejs.org/en/docs	Event loop, streams, HTTP module	Month 1
12-Factor App	12factor.net	Config, stateless processes, port binding	Month 2
Redis Documentation	redis.io/docs	All data types, BRPOP, Streams, Pub/Sub	Month 2
Docker Get Started	docs.docker.com/get-started	Containers, compose, networking	Month 2
KodeKloud Free K8s	kodekloud.com	Kubernetes fundamentals hands-on	Month 3
Kubernetes Docs	kubernetes.io/docs	Object reference, patterns, examples	Month 3
Terraform Learn	developer.hashicorp.com/terraform	IaC from basics to modules	Month 4
Prometheus Docs	prometheus.io/docs	Metrics, PromQL, alert rules	Month 5
OSTEP	ostep.org	OS internals: processes, memory, files	Parallel
NeetCode.io	neetcode.io	DSA: structured roadmap + video solutions	Daily
System Design Primer	github.com/donnemartin	System design from URL shortener to Twitter	Month 4+

Resource	URL	What For	When
GitHub Actions Docs	docs.github.com/actions	CI/CD pipeline reference	Month 5

5.2 Tools to Install

Tool	Purpose	Install Command
Docker Desktop	Container runtime + compose	Download from docker.com/products/docker-desktop
kubectl	K8s CLI	brew install kubectl / apt install kubectl
minikube	Local K8s cluster	brew install minikube / minikube start
k9s	K8s terminal UI (game changer)	brew install k9s
Terraform	IaC	brew install terraform / tfenv install
AWS CLI v2	AWS management	aws.amazon.com/cli + aws configure
helm	K8s package manager	brew install helm / snap install helm
k6	Load testing	brew install k6 / snap install k6
Lens / OpenLens	K8s GUI IDE	k8slens.dev — best K8s visualization tool
Postman / Insomnia	API testing	postman.com (free tier sufficient)
DBeaver	DB GUI for PostgreSQL	dbeaver.io (free community edition)
redis-cli	Redis CLI client	Bundled with Redis: brew install redis

5.3 Daily Schedule

Time Slot	Activity
6:00 – 8:30 AM	DEEP WORK — New concepts, project coding, LeetCode hard problems. Phone off. No messages. Highest-leverage time.
9:00 – 3:00 PM	College classes. OS and DBMS theory maps directly to your project.
3:30 – 4:30 PM	LeetCode (1 problem, 45–60 min). Track it: date, problem, difficulty, pattern, time taken.
5:00 – 6:00 PM	Rest, food, walk. Non-negotiable. No screens. You will burn out without this.

Time Slot	Activity
8:00 – 9:30 PM	Light project work — documentation, debugging, reviewing concepts, writing learning-log.md.
9:30 PM	Sleep. Non-negotiable. Sleep-deprived learning is 40% less effective.
Saturday	4–5 hrs: System design study + build a major project feature.
Sunday	2–3 hrs: Revision, mock interviews, weekly review. 1 hr rest.

5.4 Month-by-Month Milestone Tracker

Month	Must Complete
Month 1	REST API live locally. POST/GET /tasks working. PostgreSQL connected. JWT auth done. First 30 LeetCode problems.
Month 2	Full CRUD API with validation + error handling. Tests written. Swagger docs. Deployed to any cloud URL. 60 LeetCode total.
Month 3	Redis queue integrated. Worker service running. Task processed end-to-end locally. docker-compose up works. 90 LeetCode total.
Month 4	Containerized and deployed to cloud. Worker processing in production. First system design problem documented. 120 LeetCode total.
Month 5	Running on Kubernetes. HPA working. Retry logic + dead letter queue done. 150 LeetCode total. 5 system design docs.
Month 6	Load tested with k6. Rate limiting live. All K8s reliability features done. 175 LeetCode total. 9 system design docs.
Month 7	Prometheus + Grafana live. GitHub Actions CI/CD. v1.0.0 release tagged. 200 LeetCode total. Mock interviews started.
Month 8	Minimum 10 mock interviews. Resume finalized with quantified metrics. LinkedIn updated. README polished. You are ready.

FINAL REMINDER

Companies at 60+ LPA hire for depth, not breadth.

One project you can talk about for 45 minutes beats five projects you barely remember.

Your cloud background is the differentiator.

'I built this backend system AND deployed it on Kubernetes on AWS' is rare at your level.

The daily schedule is 5 hours. That is it. But those 5 hours have to actually happen.

The difference between 60 LPA and not is not talent.

It is whether you showed up at 6 AM on Tuesday when you did not feel like it.

SECTION 6 — QUICK REFERENCE GLOSSARY

6. Glossary of Key Terms

Term	Category	Definition
ACID	Database	Atomicity, Consistency, Isolation, Durability — PostgreSQL guarantees for transactions
BRPOP	Redis	Blocking right-pop from a list. CPU-efficient: the process sleeps until an item is available
CAP Theorem	Distributed	Can only guarantee 2 of: Consistency, Availability, Partition Tolerance in a distributed system
CIDR	Networking	Classless Inter-Domain Routing. 10.0.0.0/24 = 256 IPs. /16 = 65536 IPs
CrashLoopBackOff	Kubernetes	Pod keeps crashing and K8s backs off restarting it. Check kubectl describe pod + kubectl logs --previous
CNI	Kubernetes	Container Network Interface — plugin providing pod networking (Calico, Flannel, Cilium)
CoreDNS	Kubernetes	Internal DNS server in K8s. Resolves service-name.namespace.svc.cluster.local to ClusterIP
Dead Letter Queue	Architecture	Queue for messages that have failed all retry attempts. CloudTask: task_queue:dead in Redis
ETCD	Kubernetes	Distributed key-value store that is the K8s control plane's brain. Stores all cluster state
HPA	Kubernetes	HorizontalPodAutoscaler — scales pod replicas based on CPU/memory/custom metrics
Idempotency	Architecture	Calling an API multiple times has same effect as calling once. Critical for safe retries
IRSA	AWS	IAM Roles for Service Accounts — give K8s pods AWS permissions without long-lived access keys
KEDA	Kubernetes	Kubernetes Event-Driven Autoscaling — scale workers based on Redis queue depth, Kafka lag, etc.
kubectl	Kubernetes	CLI for K8s. kubectl get/describe/logs/exec/apply/delete. Learn this inside out
MVCC	Database	Multi-Version Concurrency Control — PostgreSQL readers don't block writers
Namespace (K8s)	Kubernetes	Virtual cluster partition. Separate prod/staging/monitoring namespaces for isolation

Term	Category	Definition
NetworkPolicy	Kubernetes	Pod-level firewall rules. Restrict which pods can talk to which. Default: all traffic allowed
OOMKilled	Kubernetes	Pod killed for exceeding memory limit. Increase limit or fix memory leak in application
P99 Latency	Observability	99th percentile latency — 99% of requests complete within this time. P50 = median
PVC	Kubernetes	PersistentVolumeClaim — request for storage. PostgreSQL uses PVC backed by AWS EBS
PromQL	Prometheus	Prometheus Query Language. rate(), histogram_quantile(), sum(by=label) are most used
RBAC	Kubernetes/Security	Role-Based Access Control — who can do what. ServiceAccounts get Roles via RoleBindings
Rolling Update	Kubernetes	Update pods one by one (maxSurge/maxUnavailable). Zero-downtime deployments
SLO/SLA/SLI	Observability	SLI: metric. SLO: target for SLI. SLA: contract. Error budget = 100% - SLO
Thundering Herd	Architecture	Many processes wake simultaneously after an event. Prevented by jitter in retry/backoff
WAL	Database	Write-Ahead Log — PostgreSQL writes changes to WAL first. Basis for replication + crash recovery